

UNIVERSIDAD NACIONAL

Programación IV

PROYECTO FINAL — VERSIÓN COMPLETA

Sistema Web de Turismo Local

*Continuación y evolución del Proyecto 1 (PoC-MVP) — Producto Funcional Completo
con CRUD Administrativo, Mapa Interactivo y Análisis de Calidad*

Curso: Programación 4

Tipo: Proyecto Final (continuación del PoC)

Valor: 15 % de la nota del curso

Profesor: Darin Mauricio Gamboa

Modalidad: Mismos grupos del Proyecto 1

Fecha de Entrega: 13 de Junio

Repositorio base: Mismo del Proyecto 1 (continuidad obligatoria)

Stack: React + Bootstrap · Spring Boot o Django

Despliegue: Nube pública (preferente Google Cloud Platform)

Calidad: SonarCloud + GitHub Actions + $\geq 70\%$ cobertura

Documento académico para uso exclusivo del curso

II Ciclo Lectivo · Universidad Nacional

Tabla de Contenido

Tabla de Contenido	3
1. Descripción General	4
2. Continuidad con el Proyecto 1 (PoC)	4
3. Objetivos de Aprendizaje	6
3.1 Objetivo general	6
3.2 Objetivos específicos	6
4. Alcance del Proyecto	6
4.1 Dentro del alcance	6
4.2 Fuera del alcance	7
5. Requerimientos Funcionales	8
5.1 RF-01 · Mapa interactivo con marcadores	8
5.2 RF-02 · Filtros, búsqueda y categorías	8
5.3 RF-03 · Rol administrador y CRUD	8
5.4 RF-04 · Sistema multi-pueblo	9
5.5 RF-05 · Autenticación heredada del PoC	9
6. Requerimientos No Funcionales	10
6.1 RNF-01 · Calidad de código (SonarCloud)	10
6.2 RNF-02 · Pruebas y cobertura	10
6.3 RNF-03 · Pipeline CI/CD evolucionado	10
6.4 RNF-04 · Experiencia de usuario y accesibilidad	10
6.5 RNF-05 · Seguridad	11
6.6 RNF-06 · Performance	11
7. Criterios de Evaluación	12
7.1 Criterios obligatorios (100 %)	12
7.2 Niveles de desempeño	12
7.3 Bonus del Proyecto Final (puntos extras)	12
7.4 Conversión a nota del curso	13
8. Entregables	14
8.1 Entregables técnicos	14
8.2 Entregables documentales	14
8.3 Entregables de presentación	14
9. Cronograma Sugerido	14
10. Reglas y Restricciones	15
11. Recursos Recomendados	16
12. Bibliografía y Referencias	16

1. Descripción General

El Proyecto Final constituye la evolución natural del Proyecto 1 (PoC) entregado previamente en el curso Programación 4. El equipo deberá tomar la base funcional construida en la Prueba de Concepto y MVP y transformarla en un producto completo y robusto que incorpore todas las funcionalidades que en el PoC se ofrecieron como bonus, además de mejoras técnicas, de calidad y de experiencia de usuario.

A diferencia del PoC Y MVP, donde se demostró la viabilidad técnica del sistema con funcionalidades mínimas, en el Proyecto Final se exige una solución profesional, completa, con módulo administrativo CRUD, mapa interactivo, filtros avanzados, sistema multi-pueblo, análisis estático de calidad mediante SonarCloud y mayor cobertura de pruebas unitarias e integración.

El proyecto consolida los aprendizajes del curso al integrar arquitectura cliente-servidor, autenticación federada con Google, persistencia en la nube, prácticas DevOps con CI/CD y aseguramiento de calidad de software.

2. Continuidad con el Proyecto 1 (PoC)

El equipo debe partir del mismo repositorio del Proyecto 1, mantener el historial de Git y evolucionar el código mediante ramas de feature y pull requests revisados. No se acepta empezar desde cero. La siguiente tabla resume las diferencias entre ambas entregas:

Aspecto	Proyecto 1 (PoC)	Proyecto Final
Enfoque	Viabilidad técnica	Producto funcional completo
CRUD admin	No requerido	Obligatorio
Mapa interactivo	Bonus (+5)	Obligatorio
Filtros y búsqueda	Bonus (+5)	Obligatorio
SonarCloud	Bonus (+5)	Obligatorio
Multi-pueblo	Bonus (+5)	Obligatorio (mínimo 3 pueblos)
Cobertura tests	≥ 50 %	≥ 70 %
Documentación	Básica	Profesional con diagramas
Valor	15 %	15 %

Continuidad obligatoria

El proyecto debe partir del mismo repositorio del PoC. Cualquier equipo que inicie desde cero recibirá una penalización del 15 % del puntaje total del proyecto.

3. Objetivos de Aprendizaje

3.1 Objetivo general

Evolucionar la Prueba de Concepto del Proyecto 1 hacia un producto funcional completo y profesional, aplicando principios de ingeniería de software moderna, prácticas DevOps avanzadas y aseguramiento de calidad de código mediante análisis estático automatizado.

3.2 Objetivos específicos

Al finalizar el Proyecto Final, el estudiante será capaz de:

- **Evolucionar** un PoC existente hacia un producto funcional completo siguiendo principios de ingeniería de software profesional.
- **Implementar** un módulo administrativo completo con operaciones CRUD protegidas por roles.
- **Integrar** mapas interactivos con marcadores geolocalizados y popups informativos.
- **Aplicar** filtros, búsqueda dinámica y categorización sobre datasets reales.
- **Configurar** análisis estático de calidad con SonarCloud integrado al pipeline CI/CD, con Quality Gate.
- **Aumentar** la cobertura de pruebas unitarias y de integración a un mínimo del 70 %.
- **Optimizar** la experiencia de usuario mediante diseño responsive, internacionalización y accesibilidad básica.
- **Documentar** decisiones arquitecturales, flujos de datos y métricas de calidad de manera profesional.
- **Aplicar** un flujo Git profesional con ramas, pull requests y revisión cruzada entre miembros del equipo.

4. Alcance del Proyecto

4.1 Dentro del alcance

- Refactorización y evolución del código base del PoC.
- Módulo administrativo con CRUD completo de lugares, pueblos y usuarios.
- Sistema multi-pueblo con mínimo tres pueblos diferentes y QR únicos.
- Mapa interactivo con marcadores geolocalizados.
- Filtros, búsqueda dinámica y categorización en frontend.
- Análisis estático automatizado con SonarCloud y Quality Gate.
- Cobertura de pruebas unitarias e integración mínima del 70 %.
- Diseño responsive validado en móvil, tablet y escritorio.
- Internacionalización (i18n) en español e inglés.
- Documentación técnica profesional con diagramas actualizados.

4.2 Fuera del alcance

- Sistema completo de pagos en línea o reservas con confirmación.
- Aplicación móvil nativa (iOS/Android), aunque PWA es valorada como bonus.
- Implementación de algoritmos de IA propios (los servicios externos sí pueden integrarse como bonus).
- Soporte multi-idioma más allá de español e inglés.

5. Requerimientos Funcionales

Todas las funcionalidades que en el PoC se ofrecieron como bonus, ahora son obligatorias. A continuación se detallan los requerimientos funcionales del Proyecto Final, agrupados por módulo:

5.1 RF-01 · Mapa interactivo con marcadores

- El sistema **debe** visualizar los lugares turísticos sobre un mapa real (Leaflet, Google Maps JS API o Mapbox GL).
- Cada lugar **debe** representarse con un marcador codificado por color según la categoría.

- Al pulsar un marcador, el sistema **debe** mostrar un popup con foto, nombre, descripción breve y botón de detalles.
- El sistema **debe** ofrecer un botón *Mi ubicación* que centre el mapa en la posición actual del usuario.
- El sistema **debe** permitir alternar entre Vista Lista y Vista Mapa.
- El mapa **debe** ser responsive en móvil y escritorio.

5.2 RF-02 · Filtros, búsqueda y categorías

- El sistema **debe** ofrecer una barra de búsqueda dinámica por nombre o descripción del lugar.
- El sistema **debe** permitir filtrar por categoría mediante chips o dropdown.
- El sistema **debe** permitir ordenar por nombre, categoría o fecha de creación.
- El sistema **debe** mostrar un conteo dinámico de resultados visibles.
- El sistema **debe** persistir los filtros aplicados al recargar la página (query params).
- Los filtros aplicados **deben** afectar simultáneamente la vista lista y la vista mapa.

5.3 RF-03 · Rol administrador y CRUD

- El sistema **debe** restringir el acceso al panel administrativo a usuarios con rol *ADMIN*.
- El administrador **debe** poder crear, leer, actualizar y eliminar lugares turísticos.
- El administrador **debe** poder gestionar el catálogo de pueblos disponibles.
- El administrador **debe** poder visualizar usuarios y modificar su rol.
- El sistema **debe** permitir cargar imágenes mediante Cloud Storage o servicio equivalente.
- El sistema **debe** registrar auditoría mínima: fecha de creación, fecha de modificación y autor.
- Las eliminaciones **deben** requerir confirmación explícita del usuario.

5.4 RF-04 · Sistema multi-pueblo

- El sistema **debe** soportar al menos tres pueblos diferentes con catálogos independientes.
- Cada pueblo **debe** contar con su propio código QR único de acceso.
- El usuario autenticado **debe** poder cambiar de pueblo desde un selector en el header.
- El administrador **debe** poder gestionar los lugares de cualquier pueblo.

5.5 RF-05 · Autenticación heredada del PoC

- Se mantiene el flujo OAuth 2.0 con Google del PoC.
- Solo se aceptan correos de Gmail (@gmail.com).

- La sesión **debe** mantenerse activa mediante token JWT o cookie segura.
- El sistema **debe** asignar rol *CLIENT* por defecto al primer login y permitir promoción manual a *ADMIN*.

6. Requerimientos No Funcionales

6.1 RNF-01 · Calidad de código (SonarCloud)

- El repositorio **debe** estar vinculado a una cuenta de SonarCloud.
- El pipeline **debe** ejecutar el análisis en cada push a *main* y *develop*.
- El sistema **debe** tener configurado un Quality Gate que falle el build si la calidad cae bajo el umbral.
- El README **debe** mostrar el badge de SonarCloud con calidad, bugs, code smells y cobertura.
- Métricas mínimas obligatorias: 0 bugs críticos, 0 vulnerabilidades, ≤ 30 code smells, cobertura ≥ 70 %, duplicación ≤ 5 %.

6.2 RNF-02 · Pruebas y cobertura

- Cobertura mínima total: **70 %** medida con JaCoCo (backend) y c8/Istanbul (frontend).
- Frontend: mínimo 10 pruebas (componentes, hooks, integración con API).
- Backend: mínimo 12 pruebas (controladores, servicios, repositorios, integración con BD en memoria).
- Pruebas de integración del flujo de autenticación con Google.
- Las pruebas **deben** ejecutarse automáticamente en el pipeline CI.

6.3 RNF-03 · Pipeline CI/CD evolucionado

- Build automatizado de frontend y backend.
- Linter (ESLint en frontend, Checkstyle/Flake8 en backend).
- Pruebas unitarias y de integración ejecutadas en cada push.
- Análisis SonarCloud con Quality Gate.
- Despliegue automático a la nube en la rama *main*.
- Notificación al equipo (Slack, Discord o correo) sobre el resultado del pipeline.

6.4 RNF-04 · Experiencia de usuario y accesibilidad

- Diseño **responsive** validado en móvil, tablet y escritorio.
- Estados de carga (loaders o skeletons) en cada llamada a la API.
- Manejo gracioso de errores con mensajes amigables.
- Internacionalización (i18n) mínimo en español e inglés.
- Accesibilidad básica: labels semánticos, contraste adecuado, navegación por teclado.

- Página de error 404 personalizada heredada del PoC, refinada visualmente.

6.5 RNF-05 · Seguridad

- Comunicación obligatoria sobre **HTTPS**.
- Credenciales sensibles gestionadas mediante GitHub Secrets, nunca commiteadas.
- Validación de tokens JWT en cada endpoint protegido.
- Restricción de operaciones administrativas mediante verificación de rol en backend.
- Sanitización de inputs en formularios para prevenir XSS y SQL Injection.

6.6 RNF-06 · Performance

- Tiempo de carga inicial del frontend ≤ 3 segundos en conexión 4G.
- Tiempo de respuesta promedio de la API ≤ 500 ms en operaciones simples.
- Paginación obligatoria en listados con más de 20 elementos.
- Optimización de imágenes.

7. Criterios de Evaluación

El Proyecto Final tiene un valor del 15 % de la nota del curso. La siguiente rúbrica detalla los criterios obligatorios y los puntos extras (bonus) disponibles. Cada criterio se evalúa en cuatro niveles de desempeño: Excelente, Bueno, Aceptable e Insuficiente.

7.1 Criterios obligatorios (100 %)

#	Criterio	Peso
1	Mapa interactivo con marcadores	12 %
2	Filtros, búsqueda y categorías	10 %
3	CRUD administrativo completo	18 %
4	Sistema multi-pueblo (3 o más)	8 %
5	Análisis estático SonarCloud + Quality Gate	10 %
6	Pruebas unitarias e integración ($\geq 70\%$)	12 %
7	Pipeline CI/CD evolucionado	8 %
8	Despliegue en la nube	6 %
9	UX/UI: responsive, i18n, accesibilidad	6 %
10	Documentación técnica + README	5 %
11	Evolución desde PoC (Git profesional)	3 %
12	Presentación oral y demo en vivo	2 %
	TOTAL OBLIGATORIO	100 %

7.3 Bonus del Proyecto Final (puntos extras)

Bonus	Descripción	Puntos
B1	Agente de IA integrado (chatbot, búsqueda semántica, análisis sentimiento)	+10
B2	App móvil nativa o PWA instalable	+8
B3	Sistema de reseñas y calificaciones de usuarios	+5
B4	Notificaciones push o por correo al agregar nuevos lugares	+5
B5	Despliegue con dominio personalizado y SSL configurado	+3
B6	Integración con Google Analytics o métricas de uso	+3
	TOTAL BONUS POSIBLE	+34

7.4 Conversión a nota del curso

Puntaje obtenido	Equivalencia (15 %)	Calificación cualitativa
95 – 100	14.25 – 15.00	Excelente
85 – 94	12.75 – 14.10	Muy bueno
75 – 84	11.25 – 12.60	Bueno
65 – 74	9.75 – 11.10	Aceptable
55 – 64	8.25 – 9.60	Suficiente
0 – 54	0 – 8.10	No aprobado

Fórmula de conversión

Nota final del proyecto = $(\text{Puntaje obtenido} / 100) \times 15$. Los puntos bonus se suman al puntaje obtenido, con tope máximo del 15 % del curso.

8. Entregables

El equipo deberá entregar los siguientes artefactos al cierre del proyecto:

8.1 Entregables técnicos

- **Repositorio GitHub** con historial completo desde el PoC, ramas de feature y pull requests documentados.
- **README.md** con badges de CI, SonarCloud, cobertura y licencia.
- **Aplicación desplegada** y accesible públicamente durante la presentación.
- **Tres códigos QR físicos** (uno por pueblo) para validar el sistema multi-tenancy.
- **Reporte de SonarCloud** exportado o capturas con métricas finales.

8.2 Entregables documentales

- **Documento técnico (PDF, máximo 15 páginas)** que incluya:
 - — Resumen del PoC y descripción de la evolución al producto final.
 - — Arquitectura del sistema con diagrama actualizado.
 - — Modelo de datos completo con sus relaciones.
 - — Flujo OAuth 2.0 detallado.
 - — Capturas de la aplicación desplegada en ambos roles.
 - — Capturas del Quality Gate de SonarCloud.
 - — Reflexión final del equipo sobre el aprendizaje y dificultades.

8.3 Entregables de presentación

- **Video de demostración** de 8 a 12 minutos mostrando los flujos cliente y administrador.
- **Presentación oral** de 20 minutos por equipo con participación de todos los miembros.
- **Defensa individual** posible a discreción del profesor.

10. Reglas y Restricciones

- El proyecto **debe partir del repositorio del PoC**. Empezar desde cero implica penalización del 15 % del puntaje total.
- **Originalidad obligatoria**. Se aplican las políticas antiplagio de la Universidad Nacional.

- El uso de **IA generativa** está permitido, pero el equipo debe poder defender línea por línea cualquier código presentado.
- **Flujo Git profesional**: ramas por feature, pull requests con descripción y al menos un revisor del equipo.
- **Credenciales sensibles** deben gestionarse exclusivamente vía GitHub Secrets.
- La aplicación debe estar **públicamente accesible** durante la defensa final.
- Cualquier miembro del equipo debe poder explicar **cualquier parte** del sistema.
- **Fechas inamovibles** salvo casos justificados según el reglamento universitario.

11. Recursos Recomendados

Documentación oficial y recursos académicos sugeridos para el proyecto:

- **SonarCloud:** <https://sonarcloud.io/documentation>
- **Leaflet (mapas open source):** <https://leafletjs.com/>
- **Google Maps JavaScript API:** <https://developers.google.com/maps/documentation/javascript>
- **Mapbox GL JS:** <https://docs.mapbox.com/mapbox-gl-js/>
- **React Testing Library:** <https://testing-library.com/docs/react-testing-library/intro/>
- **Spring Boot Testing:**
<https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing>
- **JaCoCo (cobertura Java):** <https://www.jacoco.org/jacoco/>
- **i18next (React):** <https://react.i18next.com/>
- **GitHub Actions + SonarCloud:** <https://docs.sonarsource.com/sonarcloud/getting-started/github/>
- **Google Cloud Run:** <https://cloud.google.com/run/docs>

12. Bibliografía y Referencias

Formato APA, 7.ª edición.

Banks, A., & Porcello, E. (2020). Learning React: Modern patterns for developing React apps (2.ª ed.). O'Reilly Media.

Beck, K. (2003). Test-driven development: By example. Addison-Wesley Professional.

Campbell, G. A., & Papapetrou, P. P. (2013). SonarQube in action. Manning Publications.

Django Software Foundation. (2025). Django documentation. <https://docs.djangoproject.com/>

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine].
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Fowler, M. (2018). Refactoring: Improving the design of existing code (2.ª ed.). Addison-Wesley.

GitHub. (2025). GitHub Actions documentation. <https://docs.github.com/en/actions>

Google. (2025). Google Cloud Platform documentation. <https://cloud.google.com/docs>

- Google Identity. (2025). Using OAuth 2.0 for web server applications.
<https://developers.google.com/identity/protocols/oauth2/web-server>
- Hardt, D. (Ed.). (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc6749>
- Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps handbook. IT Revolution Press.
- Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.
- Mozilla Developer Network. (2025). Web Accessibility Initiative (WAI) guidelines.
<https://developer.mozilla.org/en-US/docs/Web/Accessibility>
- Otto, M., & Thornton, J. (2025). Bootstrap 5 documentation. <https://getbootstrap.com/docs/>
- Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., & Mortimore, C. (2014). OpenID Connect Core 1.0. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html
- SonarSource. (2025). SonarCloud documentation: Quality gates and code analysis.
<https://sonarcloud.io/documentation>
- Spring. (2025). Spring Boot reference documentation. VMware.
<https://docs.spring.io/spring-boot/index.html>
- Walls, C. (2022). Spring in action (6.^a ed.). Manning Publications.
- World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2.
<https://www.w3.org/TR/WCAG22/>